

Projeto integrador ADSI - FATEC

1 – Matriz Curricular

Matriz curricular do CST em Análise e Desenvolvimento de Sistemas

1º semestre	2º semestre	3º semestre	4º semestre	5º semestre	6º semestre
Projeto Integrador ADS I (40 aulas)	Projeto Integrador ADS II (40 aulas)	Projeto Integrador ADS III (40 aulas)	Projeto Integrador ADS VI (40 aulas)	Projeto Integrador ADS V (40 aulas)	Projeto Integrador ADS VI (40 aulas)
Inglês I (40 aulas)	Inglês II (40 aulas)	Inglês III (40 aulas)	Inglês IV (40 aulas)	Inglês V (40 aulas)	Inglês VI (40 aulas)
Algoritmos e Lógica de Programação (80 aulas)	Banco de Dados I (80 aulas)	Banco de Dados II (80 aulas)	Desenvolvimento para Dispositivos Móveis (80 aulas)	Inteligência Artificial e Aplicações (80 aulas)	Computação Cognitiva (80 aulas)
Engenharia de Software I (80 aulas)	Linguagem de Programação I (80 aulas)	Linguagem de Programação II (80 aulas)	Estrutura de Dados (80 aulas)	Sistemas Embarcados IoT (80 aulas)	Computação em Nuvem e Infraestrutura de Servidores (80 aulas)
Sistemas Operacionais (80 aulas)	Engenharia de Software II (80 aulas)	Redes de Computadores (80 aulas)	Estatística Aplicada (80 aulas)	Engenharia de Aplicações Web (80 aulas)	Engenharia de Interfaces Web (80 aulas)
Arquitetura e Organização de Computadores (80 aulas)	Desenvolvimento Web (80 aulas)	Gestão de projetos (80 aulas)	Padrões de Projeto de Software (80 aulas)	Qualidade e Teste de Software (80 aulas)	Visão Computacional (80 aulas)
Comunicação e Expressão (80 aulas) - AS	Compliance e Segurança da Informação (80 aulas) - AS	Empreendedorismo e Inovação (80 aulas) - AS	Interação e Experiência do Usuário (80 aulas) - AS	Inteligência de Negócios (80 aulas) - AS	Gestão e Governança da TI (80 aulas) - AS
Atividades Externas à Matriz					
Atividades Externas à Matriz					
Estágio					
E = Atividade Curricular de Extensão		AS = Aulas Assíncronas On line		10% escolha da Unidade	
(240 Horas)					
Trabalho de Graduação (TG)					
(160 Horas)					
aulas/horas semanais: 24a/20h semestrais: 480a/400h	aulas/horas semanais: 24a/20h semestrais: 480a/400h	aulas/horas semanais: 24a/20h semestrais: 480a/400h Estágio: 60 horas	aulas/horas semanais: 24a/20h semestrais: 480a/400h Estágio: 60 horas	aulas/horas semanais: 24a/20h semestrais: 480a/400h Estágio: 60 horas TG: 80 horas	aulas/horas semanais: 24a/20h semestrais: 480a/400h Estágio: 60 horas TG: 80 horas
DISTRIBUIÇÃO DAS AULAS POR EIXO FORMATIVO					
Básicas	Aulas	%	Profissionais	Aulas	%
Matemática e Estatística	80	2,8	Projeto Integrador	240	8,3
Comunicação e Língua Portuguesa	80	2,8	Gestão	240	8,3
Comunicação em Língua Estrangeira	240	8,3	Infraestrutura	400	13,9
			Tecnológicas Específicas para o curso	1600	55,6
TOTAL	400	13,9	TOTAL	2480	86,1
	2160 Horas			2880 Aulas	100,0 %
RESUMO DE CARGA HORÁRIA:					
2880 aulas à 2160 horas (atende CNCST, conforme del 86 de 2009, do CEE-SP e diretrizes internas do CPS)					
+ 160 horas de Trabalho de Graduação + 240 horas de Estágio = 2.560 horas					

2 – Disciplinas do primeiro semestre

Primeiro Semestre

Sem.	Nº	Sigla	Componente	Oferta	Quantidade de aulas semestrais					
					Presenciais		On-line		Total	Atividade Curricular de Extensão
					Sala	Lab.	Sala	Lab.		
1º	1	SIGLA	Engenharia de Software I	Presencial	40	40	-	-	80	-
	2	SIGLA	Algoritmos e Lógica de Programação	Presencial	40	40	-	-	80	-
	3	SIGLA	Sistemas Operacionais	Presencial	40	40	-	-	80	-
	4	SIGLA	Arquitetura e Organização de Computadores	Presencial	40	40	-	-	80	-
	5	SIGLA	Comunicação e Expressão	On-line	-	-	80	-	80	-
	6	SIGLA	Projeto Integrador ADS I	Presencial	40	-	-	-	40	40
	7	SIGLA	Inglês I	Presencial	40	-	-	-	40	-
Total de aulas do semestre					240	160	80	-	480	40

SIGLA – Engenharia de Software I – Oferta Presencial – Total de 80 aulas

Competências desenvolvidas neste componente (profissionais e socioemocionais)

- ▶ Demonstrar capacidade de resolver problemas complexos e propor soluções criativas e inovadoras, através de algoritmos, linguagens de programação e estruturas de dados.
- ▶ Analisar as áreas funcionais da empresa e suas necessidades em relação aos sistemas de informação, organizando e apresentando aos usuários os processos envolvidos.
- ▶ Construir artefatos de sistemas multiplataformas, aplicando os conceitos e fundamentos da engenharia de software.

Objetivos de Aprendizagem

[Apresentar uma visão geral sobre a engenharia de software, bem como capacitar na escolha de processos de software a fim de planejar, conduzir e gerenciar projetos de software. Apresentar os principais aspectos relacionados à engenharia de requisitos.]

Ementa

[Fundamentos e evolução da Engenharia de Software. Paradigmas e modelos de ciclo de vida. Etapas do processo de desenvolvimento de software. Técnicas de mapeamento para implementação de sistemas. Integração e manutenção de sistemas. Planejamento e execução de testes. Elaboração de especificações funcionais. Produção e gestão de documentação técnica ao longo do projeto.]

Metodologias Propostas

[Aulas Expositivas. Aprendizagem Baseada em Projetos/Problemas. Gamificação. Atividades em pares/grupos. Sala de Aula Invertida. Estudo de Caso Real. Rotação por Estações. Coding Dojo. Participação em projetos junto aos diversos segmentos da sociedade que envolvam ações de responsabilidade social, cidadania e cultura, ciência, tecnologia e inovação.]

Instrumentos de Avaliação Propostos

[Provas, seminários, desenvolvimento de projetos, avaliação por pares, testes.]

SIGLA – Algoritmos e Lógica de Programação – Oferta Presencial – Total 30 aulas

Competências desenvolvidas neste componente (profissionais e socioemocionais)

- ▶ Demonstrar capacidade de resolver problemas complexos e propor soluções criativas e inovadoras, através de algoritmos, linguagens de programação e estruturas de dados.]

Objetivos de Aprendizagem

[Analisar e resolver problemas computacionais por meio da construção de algoritmos eficientes, utilizando técnicas de programação estruturada e modular, com apoio de linguagens de alto nível e ferramentas de desenvolvimento, promovendo a compreensão dos fundamentos da lógica de programação e sua aplicação prática.]

Ementa

[Conceitos fundamentais de algoritmos e programação. Raciocínio lógico e pensamento computacional. Tipos de dados e operadores. Estruturas básicas de controle de fluxo: sequência, decisão e repetição. Funções e modularização de programas. Introdução à recursividade. Técnicas de depuração, testes e validação de algoritmos. Representação de algoritmos em pseudocódigo, fluxogramas e linguagem de programação de alto nível. Construção de programas simples aplicados à resolução de problemas computacionais.]

Metodologias Propostas

[Aulas Expositivas. Aprendizagem Baseada em Projetos/Problemas. Gamificação. Atividades em pares/grupos. Sala de Aula Invertida. Estudo de Caso Real. Rotação por Estações.]

Instrumentos de Avaliação Propostos

[Provas, seminários, desenvolvimento de projetos, avaliação por pares, testes.]

SIGLA – Sistemas Operacionais – Oferta Presencial – Total de 80 aulas

Competências desenvolvidas neste componente (profissionais e socioemocionais)

- ▶ Avaliar os sistemas oferecidos pelo mercado e indicá-los quando necessário para a empresa. |
- ▶ Garantir segurança, integridade e performance do sistema operacional, das bases de dados e das redes utilizadas nas empresas.

Objetivos de Aprendizagem

[Compreender os conceitos e funcionalidades dos sistemas operacionais, aplicando técnicas de instalação, configuração, operação e administração de sistemas, com foco no gerenciamento de processos, memória, dispositivos e segurança, além de utilizar sistemas operacionais em ambientes corporativos e distribuições modernas para servidores]

Ementa

[Introdução aos conceitos fundamentais de sistemas operacionais, incluindo estrutura, processos, threads e sincronização de processos concorrentes. Estudo de gerenciamento de memória e memória virtual, sistemas de arquivos e gerenciamento de dispositivos. Implementação prática de sistemas operacionais em ambientes corporativos, incluindo requisitos de hardware, instalação, personalização, operação e administração de sistemas, com foco em segurança. Utilização de ambientes virtualizados e de containers com Docker, além da aplicação de linguagens de script para automação e gerenciamento de sistemas operacionais.]

Metodologias Propostas

[Aulas Expositivas. Aprendizagem Baseada em Projetos/Problemas. Gamificação. Atividades em pares/grupos. Sala de Aula Invertida. Estudo de Caso Real. Rotação por Estações.]

Instrumentos de Avaliação Propostos

[Provas, seminários, desenvolvimento de projetos, avaliação por pares, testes.]

SIGLA – Arquitetura e Organização de Computadores – Oferta Presencial – Total de 80 aulas

Competências desenvolvidas neste componente (profissionais e socioemocionais)

- ▶ Integrar os sistemas de informação da empresa otimizando o uso das bases de dados e dos recursos em rede e nuvem, aplicando os protocolos de segurança, integridade e performance do sistema operacional, das bases de dados e das redes utilizadas.]

Objetivos de Aprendizagem

[Fornecer uma visão geral de organização de computadores nos seus diversos níveis: lógica digital, microarquitetura, conjunto de instruções, sistema operacional, linguagem de programação e tecnologias.

Compreender a estrutura, operação e organização de computadores, incluindo a interação entre hardware e software, e como diferentes componentes e sistemas se integram para formar um sistema computacional eficiente.]

Ementa

[Bases numéricas e codificação de dados. Introdução à lógica digital. Conceitos básicos de arquitetura computacional incluindo as gerações de computadores, níveis de computação, processadores, canais, periféricos, modo de endereçamento, tipo de dados, conjunto de instruções, interrupções. Sistemas paralelos. Estudo da microarquitetura, interrupções e sistemas de entrada/saída (I/O). Exploração de sistemas de memória, incluindo hierarquias de memória e gerenciamento de memória. Análise de sistemas paralelos e distribuídos, além da interconexão de componentes de hardware (buses e interfaces de comunicação). Sistemas operacionais: conceitos e funções. Linguagens e ferramentas. Organização de arquivos. Arquiteturas RISC e CISC.]

Metodologias Propostas

[Aulas Expositivas. Aprendizagem Baseada em Projetos/Problemas. Gamificação. Atividades em pares/grupos. Sala de Aula Invertida. Estudo de Caso Real. Rotação por Estações.]

Instrumentos de Avaliação Propostos

[Provas, seminários, desenvolvimento de projetos, avaliação por pares, testes.]

SIGLA – Comunicação e Expressão – Oferta On-line – Total de 80 aulas

Competências desenvolvidas neste componente (profissionais e socioemocionais)

- ▶ Desenvolver comunicação interpessoal, compreensão e interpretação em situações que envolvam expressão de ideias, negociação, análise e elaboração de documentos na língua-alvo, na área de atuação profissional.
- ▶ Analisar e aplicar processos linguísticos específicos e estabelecer relações entre diferentes gêneros discursivos, com vistas à elaboração de textos que circulam no meio acadêmico e empresarial, como relatórios, laudos, pareceres e documentos técnicos.
- ▶ Comunicar-se com proficiência em língua materna.]

Objetivos de Aprendizagem

[Fundamentar situações e contextos de uso da língua escrita e falada, tendo como referência a variante padrão do português, considerando as demandas comunicacionais do meio acadêmico, corporativo e social.

Promover a leitura, análise e interpretação crítica de textos verbais e outras manifestações de linguagem, identificando tipologias e gêneros, suas especificidades e intenções discursivas.

Compreender a função da comunicação e da informação no contexto social e na área de formação do curso, desenvolvendo a capacidade de falar em público, de argumentar e de organizar ideias de modo claro, preciso, criativo e adequado às diferentes situações.

Diferenciar variantes linguísticas e aplicar a adequação da linguagem em diversos contextos, relacionando o domínio da variante padrão aos conceitos de correção linguística e hipercorreção.

Reconhecer processos linguísticos específicos e estabelecer relações entre diferentes gêneros discursivos, com vistas à elaboração de textos que circulam no meio acadêmico e empresarial, como relatórios, laudos, pareceres e documentos técnicos.

Desenvolver hábitos de análise crítica da produção textual, assegurando coerência, coesão e clareza na comunicação escrita e oral.

Refletir sobre o processo de construção da pesquisa científica, distinguindo os estatutos de cientificidade das ciências da natureza e humanas, de modo a compreender seus fundamentos epistemológicos e aplicá-los na elaboração e exposição de projetos de pesquisa.]

Ementa

[Diferenças entre oralidade e escrita. Visão geral da noção de texto. Estudo da coesão e da coerência textual, bem como dos diferentes gêneros discursivos. Discussão sobre credibilidade da informação e clareza da comunicação; compreensão e interpretação de textos; repertório cultural do leitor e construção de sentidos. Práticas de apresentação oral e argumentação. Leitura, análise e produção de textos de interesse geral, profissional e acadêmico, incluindo cartas, relatórios, e-mails, pareceres e outras formas de comunicação oral e escrita no contexto organizacional. Reflexão sobre língua e variações linguísticas, com ênfase na variante padrão, na correção e hipercorreção, e na adequação linguística em contextos profissionais. Produção de textos para as esferas empresarial e científico-acadêmica, incluindo documentos técnicos e comunicação digital. Introdução ao processo de pesquisa científica e suas classificações. Comunicação científica e ética em pesquisa (plágio e integridade acadêmica). Estudo e uso de bases de dados científicos. Métodos e técnicas de pesquisa aplicados ao contexto acadêmico e profissional. Estrutura e componentes de projetos de pesquisa, artigos, relatórios técnicos e monografias. Normas de referências e citações aplicadas à produção científica e acadêmica.]

Metodologias Propostas

[Aulas expositivo-dialogadas, apresentações orais, aprendizagem baseada em projetos/problemas, gamificação, atividades em pares/grupos, sala de aula invertida, estudo de caso real, rotação por estações.]

SIGLA – Projeto Integrador ADS I – Oferta Presencial – Total de 40 aulas

Competências desenvolvidas neste componente (profissionais e socioemocionais)

- ▶ Evidenciar o uso de pensamento crítico em situações adversas
- ▶ Identificar necessidades, desenvolver e implementar soluções, utilizando a tecnologia da informação
- ▶ Atuar com autonomia e responsabilidade, administrando conflitos, estabelecendo relações colaborativas e incentivando o trabalho em equipe

Objetivos de Aprendizagem

[Aplicar o raciocínio lógico e os fundamentos da Engenharia de Software para analisar um problema de baixa complexidade, modelar sua solução algorítmica e apresentar a documentação inicial do projeto. Desenvolver projetos que envolvam ações de responsabilidade social, cidadania, cultura, ciência, tecnologia e inovação promovendo a cooperação e troca de saberes com diversos segmentos da sociedade.]

Ementa

[Análise e decomposição de um problema do cotidiano em um contexto de TI. Elaboração de requisitos funcionais e não-funcionais. Modelagem da solução utilizando pseudocódigo e fluxogramas. Criação de um protótipo e apresentação formal da proposta.]

Metodologias Propostas

[Aulas Expositivas. Aprendizagem Baseada em Projetos/Problemas. Gamificação. Atividades em pares/grupos. Sala de Aula Invertida. Estudo de Caso Real. Rotação por Estações. Participação em projetos junto aos diversos segmentos da sociedade que envolvam ações de responsabilidade social, cidadania e cultura, ciência, tecnologia e inovação.]

Instrumentos de Avaliação Propostos

[Provas, seminários, desenvolvimento de projetos, avaliação por pares, testes.]

3 – Atividade de extensão

Detalhamento dos programas ou projetos das atividades de extensão

Título	Projeto de Desenvolvimento de Software (Fase I): Das análises à prototipação inicial.
Temática	Tecnologia da informação
Descrição	Tratar requisitos reais transformando-os em uma solução algorítmica estruturada e apresentar um protótipo de baixa fidelidade.
Objetivos	Compreender as etapas iniciais do ciclo de vida de desenvolvimento de software.
Carga horária	30h
Público-alvo	Comunidade externa, em geral (micro e pequenas empresas, ONGs, trabalhadores autônomos, instituições de natureza governamental que atendam a políticas municipais, estaduais, distrital e nacional).
Ações/Etapas de execução	Descrição do sistema; Levantamento de requisitos. Elaboração do protótipo de baixa fidelidade.
Entregas	• Protótipo de baixa fidelidade.
Instrumentos e procedimentos de avaliação	Avaliação da documentação técnica, do protótipo e da apresentação do projeto.
Componente(s) curricular(es) envolvidos	• - Projeto Integrador ADS I (30 h). • Disciplinas coadjuvantes: - Engenharia de Software I; - Comunicação de Expressão
Formas de evidência	• Resultados dos testes de usabilidade do protótipo e diário de bordo

4 – Fases do Trabalho: Projeto Integrador ADS I

4.1 Fase 0 - Divisão da turma em Grupos

4.2 Fase 1 – Definir Comunidade e Problema

Metodologia

Passo 1: Definir a comunidade externa

Objetivo: escolher a comunidade a ser estudada, que será o foco do projeto.

Público-alvo: Comunidades ou grupos que promovam responsabilidade social, cidadania, cultura, ciência, tecnologia e inovação, estimulando cooperação e troca de saberes com diferentes segmentos da sociedade. Micro e pequenas empresas, ONGs, Trabalhadores autônomos, Instituições governamentais que atendam políticas municipais, estaduais, distritais ou nacionais.

Critérios de escolha:

- Possibilidade de observar processos reais e coletar dados
- Interesse da comunidade em colaborar
- Problemas que podem ser resolvidos com **sistemas de informação simples**
- Preferência por comunidades que promovam impacto social, cultural, científico ou tecnológico, fortalecendo cooperação e troca de conhecimentos e experiências

Exemplos:

- Restaurante ou padaria local
- ONG que atende população carente
- Grupo de trabalhadores autônomos
- Escola ou instituição cultural
- Instituição governamental que implemente políticas públicas

Passo 2: Planejar levantamento de informações

Objetivo: preparar a coleta de dados de forma organizada e metodológica.

- Pesquisar e decidir técnicas de coleta:
 - Entrevistas com responsáveis
 - Questionários para usuários ou clientes
 - Observação direta de processos
 - Análise de registros ou documentos existentes
- Definir **quem será entrevistado**: gestores, funcionários, clientes ou usuários
- Elaborar roteiro de perguntas
- O objetivo do levantamento é compreender **processos, dificuldades e necessidades**

Exemplo:

Entrevistados ideais são pessoas que vivenciam diretamente o processo e podem fornecer informações relevantes sobre o funcionamento da comunidade (padaria “Pão da Vila” no exemplo). Podemos separar em grupos: Proprietário ou gerente da padaria (pode explicar os processos gerais, regras de atendimento, desafios na gestão e prioridades para melhorias.); Atendentes / garçons (vivenciam o dia a dia do registro e entrega de pedidos, podem detalhar problemas recorrentes e sugestões práticas); Cozinheiros /equipe de produção (informam sobre o fluxo interno, atrasos na preparação e dificuldades de comunicação com a frente de atendimento) ; Clientes

(podem relatar experiência de atendimento, tempo de espera, satisfação e expectativas sobre o serviço); Funcionários administrativos (se houver) (podem fornecer dados sobre registros de pedidos, estoque, faturamento e relatórios que o sistema poderia organizar).

1. **Como os pedidos dos clientes são registrados atualmente?**

Objetivo: entender o método atual (papel, planilha, memória) e identificar pontos críticos.

2. **Quais são os principais problemas ou atrasos que acontecem no atendimento?**

Objetivo: descobrir dificuldades reais que impactam clientes e funcionários.

3. **Quem é responsável por cada etapa do atendimento (anotar pedidos, preparar, entregar)?**

Objetivo: mapear funções e responsabilidades para identificar gargalos.

4. **Que tipos de informações seriam úteis para melhorar o controle de pedidos e o atendimento?**

Objetivo: levantar necessidades de dados e funcionalidades para o futuro sistema desktop.

5. **Com que frequência pedidos se perdem ou atrasam, e como isso afeta os clientes?**

Objetivo: medir impacto dos problemas atuais e priorizar soluções.

Passo 3: Coletar informações da comunidade

- Aplicar entrevistas e questionários de forma organizada

- Observar o **processo real**, não apenas relatos
- Registrar dados de forma clara (anotações, fotos, gravações, tabelas)

Exemplo: Observações realizadas:

- Cliente faz pedido → Garçon anota em papel → Cozinha prepara → Garçon entrega
- Problemas: pedidos se perdem, demora na cozinha, clientes insatisfeitos
- Registro dos dados: fotos do balcão, tabela de erros e atrasos, anotações

Passo 4: Organizar e analisar dados

- Categorizar informações coletadas:

- Processos existentes
- Problemas recorrentes
- Necessidades da comunidade

- Identificar padrões e pontos críticos

- Discussão em grupo para definir o problema central

Exemplo:

- Categorias identificadas:

- Processos existentes: atendimento manual, registro em papel
- Problemas recorrentes: pedidos perdidos, atrasos, comunicação falha
- Necessidades da comunidade: controle de pedidos, organização da fila de preparo

- Problema central definido: Falta de controle de pedidos → erros e atrasos no atendimento

Passo 5: Desenhar o problema, se possível

Objetivo: visualizar os processos e pontos de falha da comunidade.

- Mapear o processo atual da comunidade (fluxograma simples ou diagrama)
- Identificar pontos críticos usando cores ou símbolos:

- Problema crítico
- Problema médio
- Adicionar contexto:
 - Quem sofre com o problema?
 - Qual impacto no serviço ou atendimento?
 - Frequência do problema
- Validar o desenho com a comunidade para garantir precisão

Exemplo:

- Fluxograma do processo atual:
 - Cliente faz pedido → Garçom anota em papel [o pedido se perde] → Cozinha prepara, mas demora → Garçom entrega pedido atrasado
- Pontos críticos:
 - Perda de pedidos
 - Atraso na entrega
- Impacto: clientes insatisfeitos, aumento de desperdício

Passo 6: Definir problema central

- Escolher o problema mais relevante para a comunidade externa
- Deve ser passível de solução com sistema de informação

Exemplo:

- **Problema escolhido:** pedidos perdidos e atrasos no atendimento

Passo 7: Definir a ideia do sistema

Objetivo: propor **uma solução inicial de TI** para resolver o problema central.

- Pensar em como o sistema pode ajudar a comunidade
- Definir escopo geral da solução, sem entrar em requisitos detalhados

Exemplo:

- Solução proposta: sistema desktop para controle de pedidos
 - Rodará em computadores da padaria (caixa e cozinha)
 - Tela principal para registrar pedidos
 - Tela de acompanhamento do status do pedido
 - Tela de histórico e relatórios para o gerente
- Objetivo: reduzir erros, agilizar atendimento e gerar relatórios de controle
- Registrar a proposta inicial segundo documentação proposta no item 8 – Integração do Projeto Integrador com Comunicação e Expressão, para servir de base para a Fase 2 – levantamento de requisitos

4.3 Fase 2 – Proposta de Solução (Levantamento de Requisitos e Modelagem do Sistema)

Passo 8: Preparação para levantamento de requisitos

- Identificar quem vai usar o sistema
- Planejar como atender às necessidades reais da comunidade

Exemplo:

- Usuários do sistema: atendentes, cozinha, gerente da padaria

- Preparar entrevista para o levantamento de requisitos. Perguntas para levantamento de requisitos:

Requisitos Funcionais (o que o sistema deve fazer)

- Quais funções o sistema deve ter para registrar, acompanhar e finalizar pedidos?
- Objetivo: identificar funcionalidades essenciais do sistema desktop.
- Que tipos de relatórios ou informações sobre pedidos e clientes seriam úteis para o gerente ou proprietário?
- Objetivo: definir funcionalidades adicionais que atendam necessidades de controle e gestão.

Requisitos Não Funcionais (como o sistema deve funcionar)

- Em quais sistemas operacionais o sistema desktop deve funcionar (Windows, Linux, macOS) e qual é a configuração mínima de hardware necessária?
- Objetivo: garantir compatibilidade e desempenho adequado.
- Considerando que o sistema será instalado em apenas duas máquinas, qual deve ser a capacidade mínima de cada computador (processador, memória, armazenamento) para rodar o sistema de forma eficiente?
- Objetivo: definir requisitos de hardware e desempenho das máquinas que usarão o sistema.

Passo 9: Levantar requisitos do sistema

- Requisitos funcionais: o que o sistema precisa fazer
- Requisitos não funcionais: como o sistema deve funcionar. Integrar Arquitetura e Organização de Computadores e Sistemas Operacionais.

Exemplo:

- Requisitos funcionais:
 - Registrar pedidos no sistema desktop
 - Atualizar status do pedido (em preparo, pronto)
 - Emitir alertas visuais para a cozinha
 - Gerar relatórios de pedidos e desempenho
- Requisitos não funcionais:
 - Sistema rápido e confiável.
 - Interface simples, com telas claras
 - Funcionar em PCs com Windows, plataforma existente no cliente.
- Regras de Negócio - São **diretrizes ou restrições específicas da comunidade ou do processo**, que determinam **como o sistema deve operar para atender corretamente à realidade do negócio**.
 - “Um pedido só pode ser considerado pronto quando todos os itens estiverem preparados.”
 - “Cada pedido deve receber um número único sequencial.”
 - Regra: “Pedidos incompletos não podem ser entregues.” Se relaciona com o requisito funcional. : “O sistema deve bloquear a finalização de pedidos com itens pendentes.”
 - Regra exige que todos os pedidos sejam processados em tempo mínimo, o sistema precisa responder rapidamente e rodar nas máquinas existentes.
 - Regras de negócio definem “o que pode e não pode acontecer” no processo da comunidade. Requisitos funcionais descrevem como o sistema implementa essas regras. Requisitos não funcionais descrevem as condições de operação (desempenho, hardware, SO) necessárias para que o sistema funcione corretamente.

Passo 10: Modelar o sistema

- Criar pseudocódigo e fluxogramas representando processos e interações

Exemplo:

- Criar pseudocódigo para registro e atualização de pedidos
- Pseudocódigo: Sistema Desktop de Controle de Pedidos

<pre> Início do Sistema // Estrutura de dados ListaPedidos ← lista vazia Função RegistrarPedido(cliente, itens): pedidoID ← gerarID() status ← "Em preparo" pedido ← {ID: pedidoID, Cliente: cliente, Itens: itens, Status: status} adicionar pedido em ListaPedidos mostrar "Pedido registrado com sucesso. ID:", pedidoID Fim Função Função AtualizarStatus(pedidoID, novoStatus): para cada pedido em ListaPedidos: se pedido.ID = pedidoID então pedido.Status ← novoStatus mostrar "Status do pedido atualizado para:", novoStatus sair da função fim se fim para mostrar "Pedido não encontrado." Fim Função Função FinalizarPedido(pedidoID): para cada pedido em ListaPedidos: se pedido.ID = pedidoID então se todos os itens estão prontos então pedido.Status ← "Finalizado" mostrar "Pedido finalizado com sucesso." senão mostrar "Erro: alguns itens ainda não estão prontos." fim se sair da função fim se fim para Fim Função Função GerarRelatorio(): para cada pedido em ListaPedidos: mostrar "ID:", pedido.ID, " Cliente:", pedido.Cliente, " Status:", pedido.Status fim para Fim Função </pre>	<pre> // Fluxo principal Enquanto sistema está ligado: mostrar "1 - Registrar pedido" mostrar "2 - Atualizar status" mostrar "3 - Finalizar pedido" mostrar "4 - Gerar relatório" mostrar "5 - Sair" escolha ← ler entrada do usuário se escolha = 1 então cliente ← ler nome do cliente itens ← ler lista de itens RegistrarPedido(cliente, itens) senão se escolha = 2 então pedidoID ← ler ID do pedido novoStatus ← ler novo status ("Em preparo", "Pronto") AtualizarStatus(pedidoID, novoStatus) senão se escolha = 3 então pedidoID ← ler ID do pedido FinalizarPedido(pedidoID) senão se escolha = 4 então GerarRelatorio() senão se escolha = 5 então sair do sistema fim se Fim Enquanto Fim do Sistema </pre>
---	---

Passo 11: Criar protótipo

- Desenvolver telas ou layouts do sistema
- Mostrar fluxo e funcionalidades

Exemplo:

Telas:

- Tela de pedido (cliente)
- Tela de acompanhamento (cozinha)
- Tela de histórico de pedidos (gerente)

Exemplo:

Protótipo da Tela de Pedidos – Sistema Desktop

1 - Título da Tela:

- “Registro de Pedidos – Pão da Vila”

2 - Campos principais para cadastro do pedido:

- **ID do pedido:** gerado automaticamente pelo sistema (ex: 001, 002...)
- **Nome do cliente:** campo de texto
- **Itens do pedido:** lista ou tabela onde o atendente seleciona ou digita os itens (pão, bolo, café, etc.)
- **Quantidade de cada item:** campo numérico
- **Observações:** campo de texto para anotações especiais

3 - Botões de ação:

- Registrar Pedido → **adiciona o pedido à lista de pedidos**
- Cancelar → **limpa os campos**
- Atualizar Status → **abre janela para mudar o status de “Em preparo” para “Pronto”**
- Finalizar Pedido → **marca o pedido como finalizado, seguindo a regra de negócio**

4 - Lista de pedidos ativos (tabela):

- Colunas: **ID, Cliente, Itens, Quantidade, Status, Observações**
- Permite selecionar um pedido para atualizar status ou finalizar

5 - Relatórios rápidos (opcional):

- Painel lateral mostrando **número de pedidos em preparo, prontos, e finalizados**

Exemplo Visual Simplificado (ASCII / Wireframe)

```
+-----+
|          Registro de Pedidos – Pão da Vila          |
+-----+
| ID do Pedido: [ 001 ]                               |
| Nome do Cliente: [ João Silva ]                     |
| Itens do Pedido:                                     |
| [ ] Pão Frances          Qt: [ 2 ]                 |
| [ ] Bolo de Chocolate Qt: [ 1 ]                     |
| Observações: [ Sem açúcar no bolo ]                 |
+-----+
| [Registrar Pedido] [Cancelar] [Atualizar Status] |
| [Finalizar Pedido]                               |
+-----+
| Lista de Pedidos Ativos:                             |
| ID | Cliente      | Itens          | Qt | Status  |
+-----+-----+-----+-----+-----+
| 001| João Silva   | Pão, Bolo     | 3  | Em preparo|
| 002| Maria Costa | Café          | 1  | Pronto   |
+-----+-----+-----+-----+-----+
| Pedidos em preparo: 1 | Prontos: 1 | Finalizados: 0 |
+-----+-----+-----+-----+-----+
```

Passo 12: Validar solução com a comunidade

- Apresentar protótipo e lógica do sistema

- Receber feedback e ajustar solução

Exemplo:

- Apresentação do protótipo e validação com a comunidade

Após o desenvolvimento do protótipo do sistema desktop de controle de pedidos, o grupo apresentou a solução ao proprietário e aos funcionários da padaria **Pão da Vila**.

Durante a apresentação, foram mostradas as principais funcionalidades do sistema:

- Tela de registro de pedidos, onde o atendente pode inserir o nome do cliente e os itens solicitados.
- Tela de acompanhamento de pedidos, permitindo visualizar pedidos em preparo e prontos.
- Função de atualização de status, que permite marcar pedidos como “Em preparo”, “Pronto” ou “Finalizado”.
- Relatório simples de pedidos, para auxiliar o gerente no controle do atendimento.

Também foi explicado o **fluxo lógico do sistema**:

- O atendente registra o pedido no sistema.
- O pedido é salvo e aparece na lista de pedidos em preparo.
- A cozinha prepara o pedido.
- Quando estiver pronto, o status é atualizado no sistema.
- O atendente entrega o pedido ao cliente e finaliza o pedido no sistema.

Feedback recebido da comunidade

Após a apresentação do protótipo, foram coletadas opiniões dos usuários:

- O proprietário sugeriu incluir campo de observações para pedidos especiais.
- Os atendentes pediram botões maiores e mais visíveis, para facilitar o uso durante horários de movimento.
- Foi sugerido adicionar cores diferentes para cada status do pedido (ex.: amarelo para preparo, verde para pronto).

Ajustes realizados na solução

Com base no feedback recebido, foram feitas as seguintes melhorias no protótipo:

- Inclusão do campo **observações do pedido**.
- Ajuste no **layout dos botões**, tornando-os maiores e mais visíveis.
- Implementação de **cores para identificação rápida do status dos pedidos**.

Essas melhorias tornaram o sistema mais **adequado às necessidades reais da padaria**, facilitando o uso pelos funcionários e melhorando o controle dos pedidos.

Passo 13: Elaborar relatório final e apresentação

- Documentar todo o processo: comunidade, problema, requisitos, fluxogramas, pseudocódigo, protótipo e conclusões
- Preparar apresentação formal para professores e comunidade

As orientações sobre a documentação e a apresentação encontram-se no item 8 – Integração do Projeto Integrador com Comunicação e Expressão.

5- Integrar Engenharia de Software com o Projeto Integrador ADS I

Os **primeiros passos** do Projeto Integrador (comunidade, problema, coleta e análise de dados). Vamos trabalhar na solução do sistema de Informação proposto.

Projeto Integrador – Fase	Engenharia de Software 1	Observações
Fase inicial – Mapeamento dos processos correntes	—	Identificação do problema central e análise dos processos existentes sem uso de sistema de informação; define o que precisa ser resolvido
Definir ideia do sistema de informação	Projeto de software conceitual	Transformar o problema central em solução inicial de TI , de forma conceitual; serve de base para levantamento de requisitos
Levantamento de requisitos funcionais e não funcionais	Engenharia de requisitos detalhada	Definir o que o sistema deve fazer (funcionais) e como deve funcionar (não funcionais); etapa inicia após definição da ideia do sistema
Pseudocódigo / fluxograma	Modelagem detalhada do sistema	Representação da lógica e processos do sistema; planejamento para implementação
Protótipo	Planejamento e prototipagem de software	Criação de telas, layouts e fluxos de navegação; simulação da experiência do usuário
Testes do protótipo	Testes e validação de software	Verificação do funcionamento do sistema, identificação de erros e correções necessárias
Ajustes após testes	Ajustes e refinamento do software	Aprimoramento do sistema com base no feedback e nos testes realizados
Relatório final	Documentação técnica	Registro completo de todo o processo, incluindo requisitos, fluxogramas, pseudocódigo e protótipo

5.1- Integração do Projeto Integrador com Engenharia de Software I

A disciplina de **Engenharia de Software 1** apresenta conceitos fundamentais sobre o desenvolvimento de sistemas, incluindo **fundamentos, ciclos de vida, engenharia de requisitos, modelagem, testes e documentação**. No Projeto Integrador, essas mesmas etapas são aplicadas na prática, permitindo que os alunos vivenciem todo o processo de criação de um sistema de informação.

5.2 - Visão geral da Engenharia de Software

Na disciplina, os alunos aprendem:

- O que é engenharia de software;
- Como sistemas são desenvolvidos;
- Quais são as etapas de um projeto de software.

Isso se conecta com o Projeto Integrador porque vocês passam por essas etapas na prática.

Um exemplo de fluxo aplicado é:

1. Identificar o problema;
2. Levantar requisitos;
3. Planejar solução;
4. Modelar o sistema;
5. Criar protótipo.

5.3 - Integração com o Projeto Integrador

1. Resolução de problemas e proposição de soluções

- Engenharia de Software 1 desenvolve a capacidade de resolver problemas complexos e propor soluções usando algoritmos, lógica e programação.
- No projeto, problemas reais de comunidades ou organizações são analisados e soluções de sistemas de informação são propostas (ex.: controle de pedidos, cadastro de clientes).

2. Análise das áreas funcionais e levantamento de requisitos

- Na disciplina, aprende-se a analisar processos e necessidades, definindo requisitos funcionais e não funcionais.
- No projeto, entrevistas, questionários e observações permitem levantar dados concretos, que se tornam os requisitos do sistema.

3. Modelagem e construção de artefatos

- Engenharia de Software 1 aborda a criação de fluxogramas, pseudocódigo e protótipos.
- No Projeto Integrador, essas ferramentas são aplicadas para modelar o sistema, validar fluxos e facilitar a comunicação com a comunidade.

4. Testes, ajustes e documentação

- A disciplina ensina sobre planejamento de testes, manutenção e documentação técnica.
- No projeto, o protótipo é testado, ajustado conforme feedback da comunidade e documentado, garantindo a qualidade e aplicabilidade da solução.

5.4 - Conclusão

Dessa forma, o Projeto Integrador funciona como um **laboratório aplicado de Engenharia de Software 1**, permitindo que os alunos experimentem todas as fases do ciclo de vida do software, desde a identificação do problema até a entrega de uma solução funcional e documentada, consolidando competências técnicas e socioemocionais previstas na ementa da disciplina.

6 - Integração do Projeto Integrador com Algoritmos, Lógica de Programação e Estruturas de Dados

Projeto Integrador	Algoritmos e Lógica de Programação	Observação
Levantar requisitos funcionais e não funcionais	Analisar problemas computacionais e definir o que o sistema deve fazer	A definição de requisitos permite transformar necessidades reais em algoritmos e lógica de programação aplicáveis
Pseudocódigo / fluxograma	Representar processos usando pseudocódigo e fluxogramas	Ajuda a planejar a lógica do sistema, identificar fluxo de decisões, repetições e modularização
Protótipo	Construção de programas ou trechos de código simulando funcionalidades	Aplicação prática de algoritmos e estruturas de dados, demonstrando lógica e modularização
Testes do protótipo	Testar e depurar algoritmos, validar funcionamento	Permite verificar se a solução computacional atende aos requisitos e corrige falhas na lógica
Ajustes após testes	Refinamento de algoritmos e modularização do código	Aplicação de técnicas de depuração, melhoria de eficiência e clareza do programa
Relatório final	Documentação de pseudocódigos, fluxogramas e explicações dos algoritmos	Consolida o aprendizado de programação, lógica e resolução de problemas reais

A disciplina de **Algoritmos e Lógica de Programação** tem como foco o desenvolvimento do **raciocínio lógico, construção de algoritmos eficientes, modularização de programas e aplicação prática de linguagens de programação de alto nível**. O Projeto Integrador oferece um contexto real para aplicar esses conceitos, desenvolvendo soluções computacionais para problemas reais de comunidades ou organizações.

6.1 - Resolução de problemas e criatividade

- **Disciplina:** Os alunos aprendem a analisar e resolver problemas computacionais, propondo soluções criativas usando algoritmos, estruturas de dados e programação.
- **Projeto Integrador:** A identificação do problema central da comunidade leva à necessidade de propor soluções práticas de sistemas de informação (ex.: aplicativo de controle de pedidos), aplicando lógica de programação e algoritmos para resolver problemas reais de forma eficiente.

6.2 - Construção de algoritmos e pseudocódigos

- **Disciplina:** Ensina conceitos fundamentais de algoritmos, representação de processos em **pseudocódigo e fluxogramas**, controle de fluxo (sequência, decisão e repetição), funções e modularização.
- **Projeto Integrador:** Durante a modelagem do sistema, os alunos criam **fluxogramas e pseudocódigos** para representar a lógica do sistema antes da implementação. Isso garante clareza nos processos e ajuda a validar a solução junto à comunidade.

6.3 - Modularização e programação estruturada

- **Disciplina:** Introduz técnicas de modularização e organização de programas, promovendo clareza, reaproveitamento de código e manutenção.
- **Projeto Integrador:** Ao planejar a solução do sistema, os alunos podem dividir funcionalidades em módulos (ex.: cadastro de clientes, registro de pedidos, geração de relatórios), refletindo práticas de programação estruturada e modular.

6.4 - Testes, depuração e validação

- **Disciplina:** Ensina técnicas de depuração, testes e validação de algoritmos para garantir que o programa funcione corretamente.
- **Projeto Integrador:** O protótipo do sistema é testado com a comunidade, erros são corrigidos e ajustes são feitos, aplicando na prática os conceitos de validação de algoritmos e lógica de programação.

6.5 - Aplicação prática em problemas reais

- O Projeto Integrador permite aos alunos transformar **necessidades reais da comunidade em soluções computacionais**, aplicando conceitos de raciocínio lógico, algoritmos e programação aprendidos na disciplina.
- Exemplo de aplicação: identificar falhas no processo de pedidos de um restaurante e criar um algoritmo que registre, organize e notifique automaticamente a cozinha, evitando perdas e atrasos.

6.6 - Conclusão

A integração entre o Projeto Integrador e a disciplina de Algoritmos e Lógica de Programação proporciona aos alunos **uma experiência prática completa**, desde a análise do problema até a construção de soluções computacionais, consolidando competências profissionais e socioemocionais, como resolução de problemas complexos, criatividade, planejamento e trabalho em equipe.

7 - Integrar Arquitetura de Computadores e Sistemas Operacionais ao projeto

A integração de conceitos de **Arquitetura de Computadores e Sistemas Operacionais** ao projeto de software é importante para que a solução desenvolvida seja compatível com o ambiente tecnológico onde será executada.

A **arquitetura de computadores** refere-se aos componentes físicos do sistema, como processador, memória e armazenamento. Já os **sistemas operacionais** são responsáveis por gerenciar esses recursos e permitir a execução de programas.

Mesmo em projetos simples, é importante que os alunos compreendam o ambiente tecnológico disponível, evitando desenvolver soluções incompatíveis ou inviáveis.

Essa integração permite:

- compreender o ambiente tecnológico do cliente;
- garantir a viabilidade da solução proposta;
- escolher a plataforma mais adequada;
- prever limitações de desempenho e processamento;
- considerar compatibilidade entre diferentes sistemas operacionais.

7.1 - Mapeamento do ambiente do cliente

Antes de iniciar o desenvolvimento, é necessário identificar o **hardware e o software disponíveis no ambiente do cliente**. Esse levantamento evita propor soluções que não possam ser executadas nos dispositivos existentes.

Entre as principais informações que devem ser coletadas estão:

- tipo de dispositivo utilizado (computador ou celular);
- sistema operacional (Windows, Linux, Android ou iOS);
- memória e armazenamento disponíveis;
- capacidade de processamento do equipamento;
- navegador utilizado, no caso de aplicações web.

Com base nessas informações, é possível adaptar o projeto às **condições reais do ambiente tecnológico**.

7.2 - Viabilidade da solução

Um sistema/protótipo pode estar correto do ponto de vista lógico, mas ainda assim ser inviável na prática caso exija recursos que o dispositivo do cliente não possui.

Por exemplo, um software que consome grande quantidade de memória ou processamento pode apresentar lentidão ou não funcionar adequadamente em computadores mais simples.

Portanto, conhecer as características do ambiente permite **ajustar o projeto às limitações existentes**, garantindo que a solução seja funcional.

- **Hardware e arquitetura. Exemplo:** Quais computadores, tablets ou celulares os usuários irão utilizar? Qual a capacidade de memória RAM e armazenamento disponível? Existem periféricos que o sistema deve suportar (impressoras, scanners)? Qual é a qualidade da conexão de internet?
- **Software e sistemas operacionais. Exemplo:** Quais sistemas operacionais precisam ser suportados? Há restrições quanto a versões de SO ou software instalado? Existem políticas de segurança que o sistema deve respeitar?

7.3 - Relação com o protótipo, fluxogramas e UML

Ferramentas de modelagem ajudam a estruturar o projeto, enquanto a análise da arquitetura e do sistema operacional garante que ele possa ser executado no ambiente real.

Cada elemento do projeto possui um papel específico:

- **Fluxograma e pseudocódigo:** definem a lógica do sistema.
- **UML:** representa a organização e interação entre componentes.

- **Análise da arquitetura e do sistema operacional:** garante que o sistema possa ser executado no ambiente tecnológico disponível..

Dessa forma, além de definir a lógica proposta no projeto torna-se necessário verificar se o projeto é **tecnicamente viável no ambiente do cliente**.

7.4 - Escolha da plataforma

Com base nas características do ambiente identificado, os alunos devem escolher a plataforma mais adequada para implementação da solução. Entre as principais opções estão:

Aplicações Desktop – neste projeto, é a mais adequada.

São instaladas diretamente no computador do usuário e normalmente utilizadas quando o cliente trabalha principalmente com computadores.

Aplicações Web

São acessadas por meio de um navegador de internet, dispensando instalação local e permitindo acesso a partir de diferentes dispositivos.

Aplicações Mobile

São desenvolvidas para smartphones e tablets, sendo adequadas quando o uso principal ocorre em dispositivos móveis.

A escolha adequada da plataforma contribui para que o sistema seja **acessível e adequado às necessidades do usuário**.

7.5 - Considerações de desempenho e Confiabilidade

Mesmo em aplicações simples, é importante considerar o impacto do sistema nos recursos disponíveis.

Os alunos devem refletir sobre aspectos como:

- uso de memória;
- necessidade de processamento;
- volume de dados manipulados pela aplicação.
- capacidade de armazenamento disponível.

Ao considerar esses aspectos durante o planejamento, é possível evitar problemas de desempenho, como lentidão ou travamentos.

Exemplo:

Um protótipo pode ser projetado para funcionar em computadores com recursos limitados, evitando consumo excessivo de memória ou processamento.

Qual é o tempo máximo aceitável para uma operação do sistema? O sistema precisa estar disponível 24h ou em horários específicos? Como o sistema deve se comportar em caso de falhas ou quedas de energia?

7.6 - Considerações sobre Sistemas Operacionais

Além do hardware, o sistema operacional também influencia a forma como o software será organizado e executado.

Algumas práticas simples podem ser consideradas durante o desenvolvimento do projeto.

7.6.1 - Organização e armazenamento de arquivos

Uma boa organização dos arquivos do projeto facilita o desenvolvimento, a manutenção, o acesso e a atualização do sistema.

Algumas boas práticas incluem:

- criação de pastas específicas para diferentes tipos de arquivos;
- padronização de nomes de arquivos (ex.: *tela_login.png*, *dados_clientes.csv*);
- uso de formatos de arquivos compatíveis com diferentes sistemas operacionais.

Exemplo de estrutura de pastas:

```
/projeto
  /imagens
```

/dados
/prototipo
/documentacao

Essa organização torna o projeto mais claro e fácil de manter.

7.6.2- Administração básica do sistema

Também é possível refletir sobre práticas simples de administração e manutenção do sistema.

Exemplos:

- criação de rotinas de backup dos arquivos;
- organização de arquivos para facilitar atualização e manutenção do sistema.
- preparação do ambiente para futuras atualizações.
- Manter versões do protótipo e documentação, garantindo que alterações não substituam informações anteriores;

Em alguns casos, essas tarefas podem ser automatizadas por meio de **scripts simples**, utilizando ferramenta, por exemplo, como Bash ou PowerShell.

7.6.3 - Compatibilidade entre sistemas operacionais

Outro aspecto importante é verificar se a solução pode funcionar em diferentes sistemas operacionais.

Exemplo de análise:

- verificar se o protótipo pode ser executado em outros SO, evitar dependências específicas de um único sistema operacional, por exemplo, **Windows e Linux**;
- organizar arquivos e pastas de forma clara para facilitar a **portabilidade do sistema**.

Essas práticas contribuem para a **portabilidade do software**, permitindo que ele seja executado em diferentes ambientes.

7.6.4 - Considerações de Segurança

Mesmo em protótipos simples, é importante pensar em **segurança básica** para proteger dados e garantir confiabilidade da solução.

Pontos a considerar:

- **Proteção de arquivos e dados**
 - Evitar armazenar informações sensíveis sem proteção;
 - Utilizar **recursos de permissões do sistema operacional** para controlar quem pode acessar, modificar ou excluir arquivos;
 - Em planilhas ou arquivos de teste, usar **dados fictícios** para não comprometer informações reais.
- **Senhas e acesso**
 - Se houver login no protótipo, utilizar senhas simples, mas não deixar campos abertos com dados reais;
 - Explicar como a senha ou dados de usuários são tratados no protótipo.
- **Backup e recuperação**
 - Criar cópias de segurança simples dos arquivos do projeto para evitar perda de dados;
 - Manter versões do protótipo e da documentação.
 -

7.6.5- Sugestão de perguntas e possíveis respostas

7.6.5.1 - Contexto do projeto

Qual problema o seu projeto pretende resolver? O projeto pretende resolver o problema de organização de horários em uma barbearia. Atualmente os clientes marcam horários por mensagem ou pessoalmente, o que pode gerar conflitos de agenda. O sistema permitirá registrar clientes e agendar horários de atendimento.

Quem são os usuários do sistema? Os usuários do sistema serão o dono da barbearia e os funcionários que realizam os atendimentos. Eles irão cadastrar clientes e registrar os horários de atendimento.

Em que ambiente o sistema será utilizado? O sistema será utilizado dentro da barbearia, principalmente no computador do caixa, onde o dono registra os horários dos clientes.

7.6.5.2 - Ambiente tecnológico do cliente

Qual dispositivo será utilizado para acessar o sistema? Computador desktop da recepção da barbearia.

Qual sistema operacional está instalado nesse computador? Windows 10, versão 22H2.

Qual é a quantidade de memória RAM disponível? 4 GB.

Qual é o processador do computador? Intel Core i3.

Qual é a capacidade de armazenamento disponível? 256 GB SSD.

Existem limitações ou restrições do ambiente que devem ser consideradas? O computador é simples, então o sistema não pode exigir muita memória ou processamento.

7.6.5.3 - Viabilidade da solução

O protótipo que você desenvolveu funciona no computador do cliente? Sim, o protótipo é simples e roda sem problemas no computador disponível.

Se o protótipo exigisse mais recursos, quais problemas poderiam ocorrer? O sistema poderia ficar lento ou travar, já que o computador tem apenas 4 GB de RAM e processador básico.

7.6.5.4 - Relação com o protótipo

O protótipo representa as principais funcionalidades do sistema? Sim. O protótipo apresenta telas de cadastro de clientes, agendamento de horários e visualização da agenda diária. Essas são as principais funcionalidades do sistema.

Como o protótipo ajuda o cliente a entender o sistema? Mostra como o usuário irá interagir com o sistema e facilita visualizar o fluxo das operações. O protótipo mostra como serão as telas do sistema e como o usuário irá interagir com elas. Isso ajuda o cliente a visualizar como o sistema funcionará antes da implementação.

O protótipo foi pensado para funcionar no ambiente do cliente?

Sim, foi projetado para rodar em um computador desktop com poucos recursos, sem depender de internet ou nuvem.

7.6.5.4 - Escolha da plataforma

Qual plataforma você escolheu para o sistema? “Desktop”

Por que você escolheu essa plataforma? Porque o cliente possui apenas um computador, não tem serviços em nuvem, e o protótipo precisa ser simples de usar.

O computador disponível no cliente suporta a plataforma escolhida? Sim, o computador possui Windows 10, 4 GB de RAM e processador Intel Core i3, suficiente para rodar o protótipo.”

Quais seriam os problemas se você escolhesse outra plataforma, como Web ou Mobile? Web exigiria servidor ou internet confiável, que a barbearia não possui. Mobile exigiria instalação de aplicativos em smartphones, complicando o uso para o cliente.

Como a escolha da plataforma impacta a simplicidade do protótipo? Ao escolher Desktop, o protótipo roda diretamente no computador do cliente, sem precisar de configuração de rede ou servidor, sendo mais simples de usar.

7.6.5.5 - Desempenho

O sistema manipula muitos dados? Não. O sistema armazenará apenas informações básicas de clientes e horários de atendimento.

O sistema exige muito processamento? Não. O sistema realiza apenas operações simples de cadastro e consulta de horários.

O protótipo foi pensado para recursos limitados? Sim. As telas foram projetadas de forma simples para evitar uso excessivo de memória ou processamento.

7.6.5.6 - Organização do projeto

Como os arquivos do projeto estão organizados? Os arquivos foram organizados em pastas separadas para facilitar o acesso e a manutenção do projeto.

Quais pastas existem no projeto?

Exemplo:

/projeto

 /imagens

 /prototipo

 /documentacao

Existe padrão de nomes? Sim. Os arquivos possuem nomes claros e que identificam o conteúdo, como tela_login.png e tela_agendamento.png.

7.6.5.-. Compatibilidade e Portabilidade

O sistema poderia funcionar em outros sistemas operacionais? Qual ? Não garantido, pois foi desenvolvido para Windows. Por exemplo, arquivos podem não abrir corretamente em outro sistema sem adaptações.

7.6.5.8 - Segurança básica

O sistema possui login? Sim. O protótipo inclui login simples para administrador.

O sistema armazena dados sensíveis? O sistema armazena apenas dados básicos dos clientes, como nome e telefone.

Existe alguma estratégia simples para proteger os dados? No protótipo são utilizados dados fictícios para evitar exposição de informações reais. Os arquivos do protótipo estão organizados em pastas, que são protegidas por permissões de sistema operacional.

7.6.5.9 - Pergunta final de reflexão

Como Arquitetura de Computadores e Sistemas Operacionais ajudaram no projeto?

Ajuda a entender os recursos do computador do cliente e permite criar um protótipo que funcione no ambiente real, sem exigir hardware ou software que ele não possui.”

7.7 Resumo:

Integrar conceitos de **Arquitetura de Computadores e Sistemas Operacionais** ao projeto permite que os alunos desenvolvam soluções **realistas, compatíveis e adequadas ao ambiente tecnológico do cliente**, mesmo em projetos simples de primeiro semestre.

8 - Integração do Projeto Integrador com Comunicação e Expressão

Projeto Integrador – Fase	Comunicação e Expressão	Observações
Documentação do projeto (relatório técnico)	Produção de textos acadêmicos e técnicos	Redação clara, coerente e coesa; estruturação das informações do projeto; uso de normas acadêmicas e científicas
Fundamentação científica	Pesquisa científica, análise crítica e referência de fontes	Sustentação da solução com dados confiáveis; uso de bases de dados; reflexão sobre cientificidade e credibilidade das informações
Apresentação oral do projeto	Prática de apresentação, argumentação e comunicação oral	Desenvolvimento de habilidades de exposição, clareza, domínio do conteúdo e adaptação da linguagem ao público
Ajustes da documentação	Revisão textual e adequação linguística	Melhoria da coesão, coerência, precisão e estilo acadêmico do relatório
Relatório final consolidado	Documentos técnicos e científicos completos	Integração de todos os artefatos produzidos (fluxogramas, pseudocódigos, protótipo, requisitos) em um documento formal e padronizado

A disciplina de **Comunicação e Expressão** foca em desenvolver competências de comunicação escrita e oral, produção de textos técnicos e científicos, interpretação crítica de textos e apresentação de ideias de forma clara, coerente e adequada ao contexto acadêmico e profissional. No Projeto Integrador, essas competências são aplicadas de forma prática, principalmente na **documentação técnica e na apresentação do projeto à comunidade e professores**.

8.1 - Produção de documentação técnica e científica

- **Disciplina:** Desenvolve a capacidade de elaborar textos acadêmicos e técnicos, incluindo **relatórios, pareceres e documentos científicos**, respeitando normas de escrita, coesão, coerência e estilo acadêmico.
- **Projeto Integrador:** Toda a documentação do projeto (levantamento de dados, análise de problemas, fluxogramas, pseudocódigos, protótipos, requisitos do sistema) é organizada em **relatório técnico**.
- **Integração prática:** Aplicação de normas acadêmicas, escrita clara e objetiva, estruturação de informações complexas de forma compreensível, garantindo credibilidade e clareza da solução proposta.

8.2 - Apresentação oral e argumentação

- **Disciplina:** Capacita o aluno a **apresentar oralmente projetos, argumentar, organizar ideias e se comunicar de forma eficaz**, considerando o público e a situação.
- **Projeto Integrador:** Os alunos apresentam o protótipo e a solução à comunidade e aos professores, explicando decisões, processos e resultados.
- **Integração prática:** Desenvolve habilidades de argumentação, adaptação da linguagem, clareza na exposição e domínio do conteúdo técnico e científico do projeto.

8.3 - Pesquisa científica e análise crítica

- **Disciplina:** Ensina métodos de pesquisa, uso de bases de dados, classificação de conhecimento científico e reflexão sobre a cientificidade das informações.
- **Projeto Integrador:** Permite que os alunos **fundamentem o problema e a solução com dados concretos**, utilizem referências confiáveis e elaborem uma abordagem científica para o desenvolvimento do sistema.
- **Integração prática:** Sustenta o projeto em evidências, melhora a qualidade do relatório e da apresentação, promovendo rigor acadêmico.

8.4 - Escrita técnica e adequação linguística

- **Disciplina:** Desenvolve competências em **adequação linguística, correção formal e clareza** na comunicação escrita.
- **Projeto Integrador:** Toda a documentação e relatórios do projeto utilizam linguagem adequada ao contexto acadêmico e técnico, garantindo que leitores da comunidade e professores compreendam a solução.

8.5 - ESTRUTURA SUGERIDA PARA A DOCUMENTAÇÃO TÉCNICA DO PROJETO INTEGRADOR

1. Capa

- Nome do projeto
- Nome da instituição e curso
- Professor orientador
- Nome dos integrantes do grupo
- Cidade
- Data de entrega

2. Sumário

- Índice das seções com páginas

3. Introdução

- Contextualização do projeto
- Objetivos gerais e específicos
- Breve descrição da comunidade ou organização estudada
- Justificativa da escolha do problema
- Contextualização do tema
- Estrutura do trabalho

4. Fundamentação Teórica e Científica (Comunicação e Expressão)

Base conceitual que fundamenta o projeto e orienta as decisões metodológicas e técnicas adotadas.

Inclui revisão de literatura, referências acadêmicas e dados relacionados à comunidade analisada, permitindo justificar as escolhas de métodos, processos e ferramentas utilizados no desenvolvimento do projeto.

Pode incluir conceitos sobre:

- **Engenharia de Software**
(ex.: ciclo de vida de software, engenharia de requisitos, prototipagem)
- **Algoritmos e Lógica de Programação**
(ex.: algoritmos, pseudocódigo, fluxogramas)
- **Arquitetura de Computadores**
(ex.: dispositivos, memória, armazenamento)
- **Sistemas Operacionais**
(ex.: compatibilidade, gerenciamento de recursos, confiabilidade)

- **Sistemas de Informação**
(ex.: uso de sistemas em organizações)

5. Metodologia

Explica como o trabalho foi realizado.

a) Tipo de pesquisa

- Pesquisa aplicada
- Estudo de caso

b) Coleta de dados

- Entrevistas
- Questionários
- Observação

c) Etapas do projeto

- Análise da comunidade
- Definição do problema
- Levantamento de requisitos
- Modelagem do sistema
- Prototipagem
- Testes e ajustes

6. Desenvolvimento

Aqui entra toda a parte prática do projeto.

6.1 Levantamento do problema

- Descrição da comunidade
- Descrição dos processos atuais sem sistema de informação
- Identificação de pontos críticos e dificuldades
- Evidências coletadas: entrevistas, questionários, observações e registros existentes

6.2 Definição da ideia de solução

- Descrição do sistema de informação proposto
- Objetivos da solução e benefícios esperados
- Escopo inicial do sistema

6.3 Levantamento de requisitos (Engenharia de Software)

Requisitos funcionais

O que o sistema deve fazer.

Exemplos: cadastrar clientes, registrar pedidos, gerar relatórios.

Requisitos não funcionais

Como o sistema deve funcionar, incluindo aspectos como:

- **Interface e usabilidade** – sistema intuitivo e fácil de usar
- **Sistemas Operacionais** – compatibilidade, versões suportadas, políticas de segurança e garantia de funcionamento
- **Arquitetura e Organização de Computadores** – tipos de dispositivos utilizados, memória, armazenamento, periféricos, conexão com internet e tempo de resposta esperado

Prioridade e restrições

- Identificação dos requisitos mais críticos
- Limitações técnicas ou operacionais do sistema

6.4 Modelagem do sistema

- Fluxogramas dos processos do sistema

- Pseudocódigos das funcionalidades principais
- Diagramas UML (se aplicável: casos de uso, sequência, ...)

6.5 Protótipo

- Layout de telas ou interfaces
- Descrição do fluxo de navegação
- Exemplos de entradas e saídas do sistema

6.6 Testes e validação

- Planejamento e execução dos testes
- Resultados observados
- Problemas encontrados e soluções aplicadas

6.7 Ajustes e refinamentos

- Mudanças realizadas após testes e feedback da comunidade
- Melhorias na usabilidade, lógica e desempenho do sistema

7. Conclusão

- Síntese do aprendizado do grupo
- Avaliação da solução desenvolvida
- Benefícios observados para a comunidade
- Sugestões para melhorias futuras

8. Referências

- Todas as fontes consultadas, seguindo normas da **ABNT** ou outro padrão exigido pela instituição

9. Anexos

- Questionários aplicados
- Fotos ou registros do levantamento
- Documentos complementares relacionados ao projeto

8.6 - ESTRUTURA SUGERIDA PARA A APRESENTAÇÃO

Dicas de clareza

- Utilizar **slides visuais**, com fluxogramas, telas do protótipo, imagens e gráficos.
- Evitar textos longos; utilizar **tópicos curtos**.
- Explicar o **protótipo passo a passo**, como se estivesse guiando o usuário.
- Destacar o **impacto social ou organizacional da solução**.

1. Slide de abertura

- Nome do projeto
- Nome da instituição e curso
- Nome dos integrantes do grupo
- Professor orientador
- Data da apresentação

2. Contextualização do projeto

- Breve descrição da comunidade ou organização estudada
- Importância do projeto e relevância social ou organizacional
- Objetivo geral do projeto

3. Identificação do problema

- Problemas observados nos processos atuais
- Evidências coletadas (ex.: fotos, gráficos, tabelas, observações)
- Impacto do problema na comunidade

4. Ideia de solução

- Descrição do sistema de informação proposto
- Objetivos da solução
- Benefícios esperados para a comunidade ou organização
- Escopo inicial do sistema

5. Levantamento de requisitos

Requisitos funcionais

O que o sistema deve fazer.

Exemplos:

- cadastrar clientes
- registrar pedidos
- gerar relatórios

Requisitos não funcionais

Como o sistema deve funcionar, incluindo:

- **Interface e usabilidade** – sistema intuitivo e fácil de usar
- **Sistemas Operacionais – Exemplo:**
 - compatibilidade e versões suportadas
 - políticas de segurança
 - confiabilidade do sistema
- **Arquitetura e Organização de Computadores. Exemplo:**
 - tipos de dispositivos utilizados
 - memória RAM e capacidade de armazenamento
 - periféricos utilizados
 - conexão com internet
 - tempo de resposta esperado

Prioridades e restrições

- Destaque dos requisitos mais importantes
- Limitações técnicas ou operacionais identificadas

6. Modelagem do sistema

- Fluxogramas simplificados do sistema
- Pseudocódigo ou lógica resumida das funcionalidades principais
- Diagramas do sistema (se aplicável)

7. Protótipo do sistema

Apresentação visual da solução desenvolvida.

- Layouts ou telas do protótipo
- Explicação do fluxo de uso do sistema
- Principais funcionalidades implementadas

(Esta é uma das partes mais importantes da apresentação.)

8. Testes e validação

- Metodologia de testes aplicados
- Resultados obtidos
- Ajustes realizados com base nos testes

- Aprendizado obtido com o feedback da comunidade

9. Fundamentação teórica

- Breve menção às referências acadêmicas utilizadas
- Conexão com conceitos das disciplinas, como:
 - Engenharia de Software
 - Algoritmos e Lógica de Programação
 - Comunicação e Expressão
 - Arquitetura de Computadores
 - Sistemas Operacionais

10. Conclusão

- Síntese dos resultados alcançados
- Benefícios do sistema proposto
- Contribuição para a comunidade
- Possíveis melhorias ou etapas futuras

11. Agradecimentos e perguntas

- Agradecimento à comunidade e aos professores
- Espaço para perguntas e interação com a plateia

8.6 - ESTRUTURA SUGERIDA PARA A APRESENTAÇÃO DO PROTÓTIPO

1. Introdução do protótipo

Explicar que o protótipo representa o **sistema desenvolvido para resolver o problema identificado**.

Exemplo:

Sistema de controle de pedidos para reduzir erros e atrasos no atendimento.

2. Visão geral do sistema

- Apresentar uma tela inicial ou diagrama simplificado do sistema.
- Destacar os principais módulos ou funcionalidades.

Exemplo:

- cadastro de clientes
- registro de pedidos
- geração de relatórios

3. Fluxo de uso do sistema

Explicar passo a passo como o usuário interage com o sistema.

Pode ser apresentado com **fluxograma ou sequência de telas**.

Exemplo:

1. Cliente faz pedido
2. Pedido é registrado no sistema
3. Cozinha recebe a solicitação
4. Pedido é preparado e entregue

4. Funcionalidades principais

Apresentar **3 a 5 funcionalidades principais**.

Para cada funcionalidade:

- mostrar a tela do protótipo
- explicar o que ela faz
- destacar o benefício para o usuário

Exemplo:

- cadastro de clientes
- registro de pedidos
- controle de pedidos em andamento

5. Demonstração prática (opcional)

Se possível:

- mostrar o protótipo funcionando
- simular o uso do sistema

Explicar:

- entradas do sistema
- processamento
- saídas geradas

6. Feedback e ajustes

Apresentar:

- testes realizados com usuários ou comunidade
- ajustes feitos após o feedback
- melhorias implementadas

7. Conclusão do protótipo

Reforçar:

- como o sistema resolve o problema identificado
- benefícios práticos do sistema
- impacto positivo para a comunidade ou organização

9 - RESUMO

9.1 - Projeto Integrador – Integração das Disciplinas

Projeto Integrador – Fase	Engenharia de Software 1	Algoritmos / Lógica de Programação	Comunicação e Expressão	Arquitetura de Computadores / Sistemas Operacionais	Observações
Fase inicial – Mapeamento dos processos correntes	—	—	Levantamento de dados e registros; coleta de informações da comunidade	—	Identificação do problema central sem uso de sistema de informação
Definir ideia do sistema de informação	Projeto de software conceitual	—	Descrição da solução de forma clara	—	Transforma o problema em solução inicial de TI; base para levantamento de requisitos
Levantamento de requisitos	Engenharia de requisitos detalhada	Definição de funcionalidades que serão implementadas em algoritmos	—	Arquitetura: tipos de dispositivos, memória, armazenamento, periféricos, conexão de internet, tempo de resposta rápido. Sistemas Operacionais: compatibilidade, versões, políticas de segurança, garantia de funcionamento	Requisitos funcionais (o que o sistema faz) e não funcionais (como funciona); alunos elaboram perguntas considerando ambiente de execução real
Pseudocódigo / fluxograma	Modelagem detalhada do sistema	Representação da lógica, sequências e modularização do sistema	—	—	Planejamento da implementação; auxilia na construção do protótipo
Protótipo	Planejamento e prototipagem de software	Implementação inicial de algoritmos e funcionalidades	Comunicação visual do protótipo, descrição clara das telas	Teste de compatibilidade com hardware, memória e SO	Demonstração prática do funcionamento; integra lógica,

Projeto Integrador – Fase	Engenharia de Software 1	Algoritmos / Lógica de Programação	Comunicação e Expressão	Arquitetura de Computadores / Sistemas Operacionais	Observações
			e fluxo		interface e ambiente real
Testes do protótipo	Testes e validação de software	Teste da lógica de programação, depuração e validação de funcionalidades	Comunicação dos resultados e ajustes necessários	Avaliação de desempenho, consumo de recursos e confiabilidade do sistema	Ajustes baseados no feedback da comunidade
Ajustes após testes	Refinamento do software	Aprimoramento de algoritmos e modularização	Revisão textual, documentação de mudanças	Ajustes considerando limitações de hardware/SO	Melhoria de desempenho, clareza e experiência do usuário
Relatório final	Documentação técnica	Pseudocódigo, fluxogramas e trechos de código	Relatório acadêmico estruturado, fundamentação científica, normas de referência	Registro de requisitos de hardware, memória, compatibilidade de SO e desempenho	Consolida todos os artefatos do projeto, com linguagem adequada e científica
Apresentação do projeto	—	Demonstração resumida da lógica e funcionalidades	Apresentação oral clara, argumentação, comunicação visual do protótipo	Destaque de requisitos de hardware/software e compatibilidade	Demonstração prática do protótipo, impactos e benefícios

9.2. Estrutura resumida da documentação técnica – Projeto Integrador

**** documentação detalhada em " 8 Integração do Projeto Integrador com Comunicação e Expressão".**

1. **Capa:** Nome do projeto, nome da instituição e curso, professor orientador, integrantes, cidade, data (Identificação do trabalho e responsáveis)
2. **Sumário:** Índice das seções com números de página (Mapa do conteúdo do relatório)
3. **Introdução:** Contexto do projeto, objetivos gerais e específicos, descrição da comunidade, justificativa do problema, estrutura do trabalho (Apresenta o tema, objetivo e relevância do projeto)
4. **Fundamentação teórica e científica:** Base conceitual, revisão de literatura, dados da comunidade, justificativa de métodos e ferramentas; incluir conceitos de Engenharia de Software, Algoritmos e Lógica de Programação, Arquitetura de Computadores, Sistemas Operacionais, Sistemas de Informação (Explica os conceitos e teorias que sustentam o projeto)
5. **Metodologia:** Tipo de pesquisa, coleta de dados (entrevistas, questionários, observação), etapas do projeto (análise da comunidade, definição do problema,

levantamento de requisitos, modelagem, prototipagem, testes e ajustes) (Descreve como o projeto foi planejado e executado)

6. **Desenvolvimento:**

- **Levantamento do problema:** Descrição da comunidade, processos sem sistema, pontos críticos, evidências coletadas (Mostra a situação atual e os problemas identificados)
 - **Definição da ideia de solução:** Sistema proposto, objetivos e benefícios, escopo inicial (Apresenta a solução inicial planejada)
 - **Levantamento de requisitos:** Funcionais (o que faz), não funcionais (como o sistema deve funcionar, incluindo integração com SO e Arquitetura de Computadores.), prioridades e restrições (Define detalhadamente o que o sistema deve fazer e como deve funcionar)
 - **Modelagem do sistema:** Fluxogramas, pseudocódigos, diagramas (casos de uso, sequência, ERD) (Representa visualmente os processos e lógica do sistema)
 - **Protótipo:** Layouts, fluxo de telas, entradas e saídas (Demonstra a interface e funcionamento do sistema)
 - **Testes e validação:** Planejamento, execução, resultados, problemas e soluções (Avalia o funcionamento do sistema e identifica melhorias)
 - **Ajustes e refinamentos:** Melhorias após feedback, usabilidade, lógica e desempenho (Documenta alterações feitas para otimizar o sistema)
7. **Conclusão:** Síntese do aprendizado, avaliação da solução, sugestões futuras (Resume os resultados, aprendizados e próximos passos)
8. **Referências:** Fontes consultadas, normas ABNT ou padrão exigido (Lista de todos os materiais utilizados para fundamentar o trabalho)
9. **Anexos:** Questionários, fotos, registros e documentos complementares (Materiais de apoio que não entraram no corpo do relatório)

9.3. Estrutura resumida para a apresentação do Projeto Integrador

**** documentação detalhada em " 8 Integração do Projeto Integrador com Comunicação e Expressão".**

Dicas gerais:

- Slides visuais, com fluxogramas, telas do protótipo, imagens e gráficos.
 - Usar tópicos curtos e claros.
 - Explicar o protótipo passo a passo, destacando impacto social ou organizacional.
1. **Abertura:** Nome do projeto, instituição, curso, integrantes, professor orientador, data.
2. **Contextualização:** Comunidade/organização estudada, importância e relevância, objetivo geral.
3. **Problema:** Processos atuais, evidências coletadas (fotos, gráficos, tabelas), impacto na comunidade.
4. **Ideia de solução:** Sistema proposto, objetivos, benefícios e escopo inicial.
5. **Levantamento de requisitos:**
- **Funcionais:** O que o sistema faz (ex.: cadastrar clientes, registrar pedidos, gerar relatórios)
 - **Não funcionais:** Como funciona, incluindo:
 - Interface e usabilidade (intuitivo e fácil de usar)
 - Sistemas Operacionais (compatibilidade, versões, políticas de segurança, confiabilidade)
 - Arquitetura de Computadores (dispositivos, memória, armazenamento, periféricos, conexão, tempo de resposta)
 - Prioridades e restrições principais
6. **Modelagem e protótipo:**

- Fluxogramas simplificados, pseudocódigo, diagramas (se aplicável)
 - Layouts, telas, fluxo de uso e principais funcionalidades
7. **Testes e validação:** Metodologia, resultados, ajustes realizados, aprendizado com feedback
 8. **Fundamentação teórica:** Referências acadêmicas e conexão com disciplinas (Engenharia de Software, Algoritmos, Comunicação, SO, Arquitetura)
 9. **Conclusão:** Resultados, benefícios para a comunidade, melhorias ou próximos passos
 10. **Agradecimentos e perguntas:** Agradecimento à comunidade e professores, espaço para perguntas

9.3.1 Estrutura resumida da apresentação do protótipo

**** documentação detalhada em " 8 Integração do Projeto Integrador com Comunicação e Expressão".**

1. **Introdução:** Objetivo do protótipo e função principal do sistema.
2. **Visão geral:** Tela inicial ou diagrama, principais módulos (cadastro, pedidos, relatórios).
3. **Fluxo de uso:** Passo a passo da interação do usuário (ex.: pedido → registro → preparo → entrega).
4. **Funcionalidades principais:** 3 a 5 funcionalidades-chave, mostrando telas e benefícios.
5. **Demonstração prática (opcional):** Uso do protótipo, entradas, processamento e saídas.
6. **Feedback e ajustes:** Testes com usuários, melhorias implementadas.
7. **Conclusão do protótipo:** Como resolve o problema, benefícios práticos e impacto positivo.